

Bodega

Cómo está hecho el código: lenguajes, arquitectura y el render 3D

1. Los lenguajes y piezas grandes

frontend (JS+React+Three.js) --> HTTP/JSON --> backend (Python) --> base de datos
Vite empaqueta el JS REST API FastAPI + SQLAlchemy SQLite (dev) / Postgres (prod)

Backend: 100% Python, con FastAPI (framework web) + SQLAlchemy (ORM, traduce objetos Python a filas de base de datos) + JWT (tokens de sesión) + Pydantic (valida la forma de los datos que entran/salen de la API).

Frontend: JavaScript con sintaxis JSX (React), empaquetado con Vite, y Three.js para el 3D. Los estilos son CSS puro, escrito a mano, sin framework de UI.

Infraestructura: Docker / docker-compose.yml para levantar los tres servicios (frontend, backend, Postgres) con un solo comando.

2. Backend — cómo está organizado

No es un solo archivo gigante, está separado por responsabilidad:

`backend/app/models.py`

Las tablas (User, Product, Movement, Element, RoomConfig) — esto **es** la base de datos: SQLAlchemy genera las tablas SQL a partir de estas clases Python.

`backend/app/layout_logic.py`

Funciones puras (sin base de datos) que traducen un mueble genérico ({type: "bins", cols: 9, rows: 8}) a sus ubicaciones concretas (C-1-1, C-1-2, ... 72 canastas). Esta lógica garantiza que el nombre de cada ubicación sea siempre el mismo en el backend y en el visor 3D.

`backend/app/layout_service.py`

Aplica cambios a la distribución (agregar/mover/borrar muebles) con una regla de negocio importante: **nunca deja una prenda con existencias sin ubicación válida**. Por eso a veces el editor bloquea un cambio.

`backend/app/inventory_service.py`

Entradas/salidas/conteos. Usa SELECT ... FOR UPDATE para que si dos personas escanean el mismo código al mismo tiempo, la segunda espere en vez de pisar el dato de la primera — control de concurrencia real de base de datos, no un truco de la app.

`backend/app/routers/`

Los endpoints HTTP (/api/auth, /api/layout, /api/products, /api/movements, /api/reports) — cada archivo es un grupo de rutas.

`backend/app/security.py, deps.py`

Login con JWT, y el interruptor `SKIP_AUTH` que se activó para desarrollo local.

3. Frontend — cómo está organizado

`pages/`

Una página por pestaña de abajo — Warehouse (el 3D), Scan, Inventory, Orders, History, Login. React Router decide cuál mostrar según la URL.

`context/AuthContext.jsx`

Guarda el token JWT y el usuario logueado, disponible para toda la app sin pasarlo manualmente por cada componente (patrón "Context" de React).

`hooks/useApi.js`

En vez de WebSockets, usa **sondeo (polling)**: cada 4-6 segundos vuelve a pedir los datos. Así, si otra persona escanea algo, tú lo ves aparecer solo en unos segundos. Es una decisión consciente de simplicidad.

`components/`

EditPanel, LocationSheet, ProductModal, etc. — piezas de UI reutilizables, cada una habla con la API mediante `api.js`.

4. El render 3D — la parte más interesante

Vive en un solo archivo, `frontend/src/three/WarehouseScene.js`, y es una **clase de JavaScript aislada de React** — React solo le pasa datos (`setLayout`, `setProducts`) y escucha eventos de toque. Nada de HTML/CSS ahí adentro, todo es Three.js puro.

No hay modelos 3D importados (ni `.glb`, ni `.obj`, nada descargado). Cada mueble se construye por código con formas geométricas primitivas de Three.js:

```
// una estanteria es, literalmente, codigo que arma cajas:
const post = new THREE.BoxGeometry(0.06, H, 0.06) // las 4 patas
const beam = new THREE.BoxGeometry(W, 0.08, 0.05) // las vigas
const deck = new THREE.BoxGeometry(W - 0.04, 0.02, D - 0.04) // el nivel
```

Por eso cuando agregas una estantería de 5 niveles en vez de 3, no carga un archivo distinto — el mismo código matemático genera 5 niveles en vez de 3.

Piezas clave del motor

1. Construcción desde datos (`_buildWorld`)

Recibe el JSON del backend (`layout.elements`) y por cada mueble llama a su función (`_buildShelf`, `_buildRack`, `_buildBins...`) que arma la geometría. Si el mundo se reconstruye (por ejemplo al agregar un mueble), Three.js libera la memoria de las formas viejas (`geometry.dispose()`) para no dejar fugas.

2. Instancing para rendimiento

Una pared de canastas puede tener 72 gavetas — en vez de crear 72 objetos 3D separados (lento), usa `THREE.InstancedMesh`: una sola geometría dibujada 72 veces con distintas posiciones, en una sola llamada a la GPU. Es la técnica estándar para repetir miles de objetos iguales sin matar el rendimiento en un celular.

3. Cámara orbital manual

No usa el `OrbitControls` de ejemplo de Three.js — está escrita a mano con coordenadas esféricas (`th` = ángulo horizontal, `ph` = ángulo vertical, `r` = distancia) que se animan suavemente hacia un "objetivo" (`goal`) cada cuadro. Así los botones "Vista general" / "Planta" / enfocar una estantería son solo "cambia el objetivo de la cámara" y se desliza solo.

4. Detección de toques (raycasting)

Cada ubicación y cada mueble tiene una caja invisible (`hitMat`, opacidad 0). Cuando tocas la pantalla, se lanza un rayo desde la cámara hasta ese punto y se pregunta "¿con qué caja invisible chocó?" — así sabe qué ubicación o mueble tocaste, sin depender de coordenadas de pantalla.

5. Realismo agregado en el pulido visual

Materiales PBR (`MeshStandardMaterial`) con `roughness`/`metalness` para que la luz rebote de forma creíble (metal vs. cartón vs. concreto).

Texturas procedurales generadas en código, no descargadas: se dibuja en un `<canvas>` invisible (con `fillRect`, ruido aleatorio, líneas) y ese canvas se usa como textura — así el piso tiene moteado de concreto y las cajas tienen líneas de cartón corrugado + cinta, sin bajar ninguna imagen de internet.

Sombras reales (`renderer.shadowMap`), con una luz direccional ajustada al tamaño exacto del cuarto, más una luz de relleno fría y luces cálidas de techo.

Tone mapping cinematográfico (`ACESFilmicToneMapping`) y niebla sutil (`THREE.Fog`) para que los colores no se vean planos.

5. En resumen

Es una arquitectura bastante estándar y sin trucos raros: **API REST clásica + base de datos relacional + un motor 3D casero hecho de formas geométricas simples, no de modelos importados**. La complejidad real no está en gráficos avanzados, sino en la lógica de negocio: no perder inventario, no chocar códigos, no pisar datos entre dos personas escaneando a la vez.

Generado a partir de la explicación de la conversación sobre el proyecto Bodega.